

UNIVERSITÉ
SORBONNE PARIS NORD

- [# Portail](#) >
- [WSAIR2_2526](#) >
- [Groupes](#) >
- [Groupe 3](#) >
- [Wiki](#) >
- [Rendu-TP4](#) >
- [Page principale](#)

- [WSAIR2_2526](#)

Web Sémantique

WSAIR2_2526

Jérôme NOBECOURT

Groupe 3

[📁 Documents et liens](#) | [🗨️ Forums](#) | [📖 Wiki](#) | [💬 Discussion](#)

Groupe 3

Wiki : Rendu-TP4

- [Retour à la page](#)
- [Editer cette page](#)
- [Historique de cette page](#)

[🏠 Page principale](#) | [🕒 Dernières modifications](#) | [📄 Liste des pages](#) | [📋 Liste des wikis](#) | [🔍 Rechercher](#)

Page principale

Compte-rendu TP4 — XML Schema, XSLT et Unicode

Web Sémantique · WSAIR2_2526 · Université Sorbonne Paris Nord

Table des matières

1. 1. Introduction

1. 2. Exercice 1 — Parlement Européen : XML Schema (XSD)

1. 3. Exercice 1 — Parlement Européen : Document XML instance
1. 4. Exercice 1 — Parlement Européen : Transformation XSLT
1. 5. Exercice 2 — Manipulation des formats Unicode
1. 6. Conclusion

1. Introduction

Ce TP porte sur deux exercices distincts couvrant des thématiques complémentaires du Web sémantique et du Web multilingue.

Le premier exercice concerne la modélisation XML des débats du Parlement Européen. Il s'agit de créer un schéma XSD décrivant la structure des transcriptions publiées sur europarl.europa.eu, de produire un document XML valide conforme à ce schéma, et enfin de transformer ce document en HTML à l'aide d'une feuille XSLT.

Le second exercice porte sur l'encodage des caractères Unicode. Il demande de programmer, en Python (en remplacement de PHP), un outil permettant de visualiser les représentations internes d'un caractère ou d'une plage de caractères en UCS-4, UTF-8 et UTF-16 BE, sans recourir à aucune librairie de conversion d'encodage.

Les fichiers produits pour ce TP sont :

- `parlement_europeen.xsd` : le schéma XML Schema ;
- `debat_20200114_point14.xml` : le document XML instance (point 14 de la séance du 14 janvier 2020) ;
- `parlement_europeen.xsl` : la feuille de transformation XSLT 1.0 ;
- `aperçu.html` : le rendu HTML obtenu après transformation ;
- `app.py` : le serveur Python pour la visualisation Unicode.

2. Exercice 1 — Parlement Européen : XML Schema (XSD)

2.1 Question préalable : pourquoi XML plutôt qu'un SGBD-R ?

L'énoncé pose cette question comme point de départ. Les raisons principales sont les suivantes. Les débats parlementaires ont une structure hiérarchique naturelle (séance > points d'ordre du jour > interventions > transcriptions), qui se modélise plus naturellement en arbre XML qu'en tables relationnelles. Par ailleurs, les données sont hétérogènes : certains champs sont optionnels selon le type d'interlocuteur (un commissaire n'a pas de groupe parlementaire, un député n'a pas forcément d'adresse e-mail publiée) ; XML tolère cette variabilité sans lignes à valeurs nulles. Enfin, les pages du site europarl.europa.eu sont générées côté serveur par transformation XSLT à partir d'une source XML — la chaîne XML/XSLT est donc native à ce domaine.

2.2 Type racine observé

L'observation des pages du site du Parlement Européen (par exemple `CRE-9-2020-01-14-ITM-014_FR.html`) montre que chaque page correspond à un point d'ordre du jour d'une séance donnée : elle contient les interventions de ce seul point, précédées d'informations sur la séance et les interlocuteurs. Le type racine choisi est donc TypeTranscriptionDebats, qui regroupe les référentiels (pays, mouvements politiques, interlocuteurs) et la séance avec ses points d'ordre du jour.

2.3 Types simples définis

Les types simples suivants ont été créés par restriction des types prédéfinis XSD :

- TypePeriodeMandature : restriction de `xs:string` par énumération des dix mandatures (de « 1979-1984 » à « 2024-2029 »).
- TypeCodePaysISO3166 : restriction de `xs:string` par expression régulière `A-Z{2}` (code ISO 3166-1 alpha-2).
- TypeCodeLangueISO639 : restriction de `xs:string` par expression régulière `a-z{2}` (code ISO 639-1).
- TypeLangueDocument : union de TypeCodeLangueISO639 et d'une restriction à la valeur littérale "origine", pour permettre d'indiquer que la transcription est dans la langue d'origine de l'orateur.
- TypeGroupeParlementaire : restriction de `xs:string` par énumération des huit groupes politiques actuels (PPE, S&D, Renew, Verts/ALE, ID, ECR, GUE/NGL, NI).

- TypeIdentifiantDepute : restriction de `xs:string` par expression régulière `0-9{4,8}`.
- TypeHeure : restriction de `xs:string` par expression régulière `(010-9|20-3):0-50-9`.
- TypeNomPaysLocalise : restriction de `xs:string` avec longueur minimale 2 et maximale 100.
- TypeEmail : restriction de `xs:string` par expression régulière `^@\s+@^@\s+\.\s+`.
- TypeURL : restriction de `xs:anyURI` par expression régulière `https?://.+`.

2.4 Types complexes définis

Les types complexes suivants ont été définis :

- TypePays (séquence) : attribut `code` de type `TypeCodePaysISO3166`, éléments `nom` et `nombreDeputes`.
- TypeMouvementPolitique (séquence) : attribut `id` de type `xs:ID`, éléments `nom`, `codePays` et `groupeParlementaire`.
- TypeDepute (all, pour permettre tout ordre) : attributs `id` (`xs:ID`) et `identifiantPE` ; éléments `prenom`, `nom`, `codePays`, `mouvementPolitiqueRef` (`xs:IDREF`), `groupeParlementaire` ; éléments optionnels `dateNaissance`, `lieuNaissance`, `email`, `siteWeb`.
- TypeMembreCommission (all) : attribut `id` (`xs:ID`) ; éléments `prenom`, `nom`, `codePays`, `fonction`.
- TypeIntervention (séquence) : attributs `id` (`xs:ID`) et `orateur` (`xs:IDREF`) ; éléments `heureDebut` et `transcription` (`TypeTranscription`).
- TypeTranscription (simpleContent extension) : contenu textuel étendu par un attribut `langue` de type `TypeCodeLangueISO639`.
- TypePointOrdreJour (séquence) : attributs `id` et `numero` ; éléments `intitule`, `heureDebut` et une ou plusieurs `intervention`.
- TypeSeanceParlementaire (séquence) : attribut `id` ; éléments `date`, `lieu` (optionnel), `mandature`, et un ou plusieurs `pointOrdreJour`.
- TypeTranscriptionDebats (type racine, séquence) : attributs `langue` et `urlSource` ; éléments `pays` (1..n), `mouvementsPolitiques`, `interlocuteurs`, `seance`.

2.5 Contraintes de clé

Quatre contraintes `xs:key` garantissent l'unicité des identifiants dans le document :

```
<xs:key name="cleDepute">
  <xs:selector xpath="//depute"/>
  <xs:field xpath="@id"/>
</xs:key>

<xs:key name="cleMembreCommission">
  <xs:selector xpath="//membreCommission"/>
  <xs:field xpath="@id"/>
</xs:key>

<xs:key name="cleMouvement">
  <xs:selector xpath="//mouvementPolitique"/>
  <xs:field xpath="@id"/>
</xs:key>

<xs:key name="cleIntervention">
  <xs:selector xpath="//intervention"/>
  <xs:field xpath="@id"/>
</xs:key>
```

Une contrainte `xs:keyref` assure que chaque `mouvementPolitiqueRef` d'un député référence bien un mouvement déclaré dans le document :

```
<xs:keyref name="refMouvementDepute" refer="cleMouvement">
  <xs:selector xpath="//depute/mouvementPolitiqueRef"/>
  <xs:field xpath="."/>
</xs:keyref>
```

La contrainte principale de l'énoncé — « toute intervention doit être attribuée à un interlocuteur identifié » — est assurée par le typage `xs:IDREF` de l'attribut `orateur` : le validateur XML rejette automatiquement tout document dont cet attribut référencerait un identifiant inexistant.

3. Exercice 1 — Parlement Européen : Document XML instance

3.1 Structure générale du fichier

Le fichier `debat_20200114_point14.xml` représente le point 14 de la séance du 14 janvier 2020, dont la source est http://www.europarl.europa.eu/doceo/document/CRE-9-2020-01-14-ITM-014_FR.html.

L'élément racine déclare sa conformité au schéma et précise la langue d'affichage :

```
<?xml version='1.0' encoding='UTF-8'?>
<?xml-stylesheet type="text/xsl" href="parlement_europeen.xsl"?>
<transcriptionDebats
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="parlement_europeen.xsd"
  langue="origine"
  urlSource="http://www.europarl.europa.eu/doceo/document/CRE-9-2020-01-14-ITM-014_FR.html">
```

Le document est organisé en quatre sections : le référentiel des pays (13 pays représentés), les mouvements politiques nationaux (un par groupe parlementaire et par pays concerné), les interlocuteurs (7 députés et 1 commissaire européen), et enfin la séance avec son unique point d'ordre du jour et les 8 interventions correspondantes.

3.2 Exemple de député

```
<depute id="d_verhofstadt" identifiantPE="00124">
  <prenom>Guy</prenom>
  <nom>Verhofstadt</nom>
  <codePays>BE</codePays>
  <mouvementPolitiqueRef>mp_mr_be</mouvementPolitiqueRef>
  <groupeParlementaire>Renew</groupeParlementaire>
  <dateNaissance>1963-04-11</dateNaissance>
  <lieuNaissance>Dendermonde</lieuNaissance>
  <siteWeb>https://www.europarl.europa.eu/meps/fr/00124</siteWeb>
</depute>
```

3.3 Exemple d'intervention

```
<intervention id="i01" orateur="d_sassoli">
  <heureDebut>15:04</heureDebut>
  <transcription langue="it">
    Colleghe e colleghi, il punto dell'ordine del giorno riguarda
    la situazione in Medio Oriente ...
  </transcription>
</intervention>
```

3.4 Validation avec xmllint

La commande suivante vérifie que le document est valide conformément au schéma :

```
xmllint schema-parlement_europeen.xsd debat_20200114_point14.xml --noout
```

Pour vérifier que la contrainte d'intégrité référentielle fonctionne correctement, il suffit de modifier l'attribut `orateur` d'une intervention pour référencer un identifiant inexistant (par exemple `orateur="d_inconnu"`). La validation échoue immédiatement avec une erreur de type IDREF.

4. Exercice 1 — Parlement Européen : Transformation XSLT

4.1 Principe de la feuille XSLT

La feuille `parlement_europeen.xsl` est une feuille XSLT 1.0 qui transforme un document XML conforme au schéma en une page HTML reproduisant l'apparence du site `europarl.europa.eu` : barre de navigation bleue (couleur institutionnelle #003399), filet jaune (#ffcc00), tableau récapitulatif des interlocuteurs en bas de page.

Trois clés XSLT sont déclarées en début de feuille pour permettre des jointures efficaces entre éléments :

```
<xsl:key name="interlocuteur" match="depute | membreCommission" use="@id"/>
<xsl:key name="pays" match="pays" use="@code"/>
<xsl:key name="mouvement" match="mouvementPolitique" use="@id"/>
```

4.2 Template principal

Le template correspondant à l'élément racine produit le squelette HTML, les métadonnées de la séance, puis délègue l'affichage des points d'ordre du jour et du tableau des interlocuteurs à des templates spécialisés :

```
<xsl:template match="/transcriptionDebats">
  ...
  <xsl:apply-templates select="seance/pointOrdreJour"/>
  <xsl:apply-templates select="interlocuteurs/depute
                                | interlocuteurs/membreCommission" mode="table"/>
  ...
</xsl:template>
```

4.3 Template d'intervention

Le template `intervention` résout dynamiquement l'orateur grâce à la clé `interlocuteur` et affiche son nom, son groupe et son pays :

```
<xsl:template match="intervention">
  <xsl:variable name="idOrateur" select="@orateur"/>
  <xsl:variable name="personne" select="key('interlocuteur',$idOrateur)"/>
  ...
  <xsl:choose>
    <xsl:when test="$personne/self::depute">
      <xsl:text>Député européen – </xsl:text>
      <xsl:value-of select="$personne/groupeParlementaire"/>
      <xsl:text>, </xsl:text>
      <xsl:value-of select="key('pays',$personne/codePays)/nom"/>
    </xsl:when>
    <xsl:otherwise>
      <xsl:value-of select="$personne/fonction"/>
    </xsl:otherwise>
  </xsl:choose>
  ...
</xsl:template>
```

L'utilisation de `xsl:choose` permet de distinguer les deux types d'interlocuteurs (député vs. commissaire) et d'afficher les informations appropriées pour chacun.

4.4 Résultat obtenu

L'application de la feuille XSLT au document XML produit le fichier `aperçu.html`. La page résultante affiche la barre institutionnelle du Parlement Européen, les métadonnées de la séance (date : 2020-01-14, lieu : Strasbourg, mandature : 2019-2024), les interventions du point 14 avec heure, nom de l'orateur, groupe et pays, et un tableau récapitulatif de tous les interlocuteurs. Le résultat est visuellement proche des pages officielles du site `europarl.europa.eu`.

5. Exercice 2 — Manipulation des formats Unicode

5.1 Choix du langage et structure générale

L'énoncé préconise PHP, mais autorise tout langage. Le choix s'est porté sur Python 3, qui dispose des opérateurs de manipulation de bits nécessaires (>>, &, |) et permet de créer facilement un petit serveur HTTP sans dépendances externes grâce au module `http.server` de la bibliothèque standard.

Conformément à la règle du jeu, aucune librairie de conversion d'encodage ou de manipulation de « wide characters » n'a été utilisée. Toutes les transformations sont programmées manuellement par opérations sur les bits.

Le fichier `app.py` se lance ainsi depuis un terminal :

```
python3 app.py
```

Un serveur HTTP démarre sur `http://localhost:8000`. L'interface propose trois modes de saisie : un caractère unique, une plage hexadécimale (par exemple `0041` à `005A` pour A–Z), ou une plage décimale.

5.2 Conversion en UCS-4

UCS-4 est l'encodage « natif » Unicode : chaque point de code est représenté sur 4 octets fixes en big-endian. La conversion consiste simplement à décomposer le point de code en quatre octets par décalages de 8 bits successifs :

```
def to_ucs4_bits(cp: int) -> str:
    b3 = (cp >> 24) & 0xFF
    b2 = (cp >> 16) & 0xFF
    b1 = (cp >> 8) & 0xFF
    b0 = cp & 0xFF
    return f"{b3:08b} {b2:08b} {b1:08b} {b0:08b}"
```

5.3 Conversion en UTF-8

UTF-8 est un encodage à longueur variable. Les règles d'encodage sont les suivantes :

- U+0000–U+007F : 1 octet, motif `0xxxxxxx`
- U+0080–U+07FF : 2 octets, motif `110xxxxx 10xxxxxx`
- U+0800–U+FFFF : 3 octets, motif `1110xxxx 10xxxxxx 10xxxxxx`
- U+10000–U+10FFFF : 4 octets, motif `11110xxx 10xxxxxx 10xxxxxx 10xxxxxx`

L'implémentation manuelle utilise les opérateurs >> (décalage à droite) et & (ET bit à bit) pour extraire les bits du point de code, et | (OU bit à bit) pour y ajouter les bits de structure UTF-8 :

```
def to_utf8(cp: int):
    if cp <= 0x7F:
        bytes_list = cp
    elif cp <= 0x7FF:
        b0 = 0b11000000 | (cp >> 6)
        b1 = 0b10000000 | (cp & 0x3F)
        bytes_list = b0, b1
    elif cp <= 0xFFFF:
        b0 = 0b11100000 | (cp >> 12)
        b1 = 0b10000000 | ((cp >> 6) & 0x3F)
        b2 = 0b10000000 | (cp & 0x3F)
        bytes_list = b0, b1, b2
    else:
        b0 = 0b11110000 | (cp >> 18)
        b1 = 0b10000000 | ((cp >> 12) & 0x3F)
        b2 = 0b10000000 | ((cp >> 6) & 0x3F)
        b3 = 0b10000000 | (cp & 0x3F)
        bytes_list = b0, b1, b2, b3
    ...
```

5.4 Conversion en UTF-16 BE

UTF-16 encode la plupart des caractères sur un seul mot de 16 bits. Les points de code supérieurs à U+FFFF (plan supplémentaire) nécessitent une paire de surrogates. Le calcul de la paire est le suivant :

- `cp' = cp - 0x10000`
- `surrogate haute : 0xD800 + (cp' >> 10)`
- `surrogate basse : 0xDC00 + (cp' & 0x3FF)`

```
def to_utf16_be(cp: int):
    if cp <= 0xFFFF:
        words = cp
    else:
        cp_prime = cp - 0x10000
        high = 0xD800 + (cp_prime >> 10)
        low = 0xDC00 + (cp_prime & 0x3FF)
        words = high, low
    bytes_list =
    for w in words:
        bytes_list.append((w >> 8) & 0xFF)
        bytes_list.append(w & 0xFF)
    ...
```

5.5 Coloration des bits de structure

Pour faciliter la compréhension visuelle, la représentation binaire distingue en rouge les bits de structure (préfixes UTF) et en bleu les bits de données (bits effectifs du point de code). Par exemple, pour le caractère « é » (U+00E9, 2 octets UTF-8) :

- Octet 1 : **110** (structure) + **00011** (données)
- Octet 2 : **10** (structure) + **101001** (données)

5.6 Interface web

L'interface affichée dans le navigateur reproduit les fonctionnalités décrites dans l'énoncé et illustrées par l'exemple de l'enseignant. Pour chaque caractère, un tableau à trois lignes présente les encodages UCS-4, UTF-8 et UTF-16 BE, chacun décliné en binaire (avec coloration), hexadécimal et décimal. Les plages sont limitées à 512 caractères pour éviter une surcharge du navigateur.

Exemple de résultat pour le caractère 😊 (U+1F600) :

Encodage de U+1F600 (😊)

	Encodage	Hexadécimal	Décimal
UCS-4	00 01 F6 00	0 1 246 0	
UTF-8	F0 9F 98 80	240 159 152 128	
UTF-16 BE	D8 3D DE 00	216 61 222 0	

6. Conclusion

Ce TP a permis de mettre en pratique deux aspects complémentaires du Web sémantique et du Web multilingue.

Pour l'exercice Parlement Européen, la conception du schéma XSD a nécessité de réfléchir aux types de données adaptés à chaque information (codes normalisés ISO, dates, heures, URL, identifiants...), aux contraintes d'intégrité référentielle entre interlocuteurs et interventions, et au choix entre séquence ordonnée et composition non ordonnée (`xs:all`) pour les types complexes. La feuille XSLT a ensuite permis de produire un rendu HTML fidèle à l'original en exploitant les clés XSLT pour réaliser des jointures entre éléments sans répétition d'information.

Pour l'exercice Unicode, l'implémentation manuelle des algorithmes d'encodage — en particulier les paires de surrogates UTF-16 et les masques de structure UTF-8 — a mis en lumière la logique interne de ces formats, que l'utilisation de bibliothèques de haut niveau tendrait à masquer. La coloration différenciée des bits de structure et des bits de données dans la sortie HTML rend cette logique immédiatement visible pour l'utilisateur.

Gestionnaire(s) de WSAIR2_2526 : Jérôme NOBECOURT

Administrateur de Portail : Webmestre

Utilise la plateforme Claroline © 2001 - 2013

modifiée depuis 2013 à aujourd'hui pour ce portail